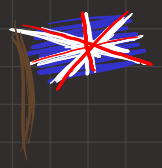
DISTRIBUTED SYSTEMS NOTES



Carlo Da Rome NOV. 5. 2026

CH. 1 G.P.E.

QUESTION 1: WHAT IS A DISTRIBUTED SYSTEM?

ANSWER: A DISTRIBUTED SYSTEM IS A SET OF COMPUTERS THAT COOPERATE TO EXECUTE OPERATIONS OR TO STORE DATAS, THEY DON'T USE CLOCK FOR SINCRONIZATION, THE SINCRONIZATION IS POSSIBLE THROUGH MESSAGES.

- A LOT OF PROBLEMS IN THIS TYPE OF SYSTEM CAN BE TRACED BACK TO G.P.E. (global predicate evaluation) THIS IS EVALUATE IF THE SYSTEM SATISFIES A CONDITION.
- AN ASINCRONOS DYSTRIBUTED SYSTEM (A.D.S.) IS A SYSTEM WITH THE FOLLOWING PROPERTIES:

1. THE M PROCESSES IN THE SYSTEM IS REPRESENTED WITH THE SET $(P_1, \dots, P_m)$.
2. THE COMMUNICATION CHANNELS IS BASICALLY NOT FIFO, THIS MEANS THAT THE ORDER OF SENDED MESSAGES COULD BE DIFFERENT TO THE RECIEVE ORDER.
3. PROCESSES ARE STRONG CONNECTED, THIS MEANS THAT PROCESSES ARE ALL CONNECTED EACH OTHER.
4. CHANNELS ARE RELIABLE... WE CAN ASSUME THAT WE CANNOT HAVE DATA LOST OR COMPROMIZED.

5. SINCRONIZATION PASSES ONLY THROUGH MESSAGES

THIS IS A THEORETICAL MODEL, IT DOEST EXIST IN REALITY 'CAUSE IS NOT POSSIBLE TO REALIZE THAT BUT IT IS A GOOD ABSTRACTION BECAUSE.

1. IS VERY SIMPLE AND DOESN'T HAVE ANY BOUND IN TERM OF SPEED OR RETARD, IF ONE SOLUTION IS GOOD ON THIS MODEL IT IS ALSO GOOD IN REAL LIFE.

2. SOME LAYERS IN REAL SYSTEMS FOLLOW THIS MODEL (RULE 4 IS IMPOSSIBLE TO SATISFIES IN REAL LIFE)

- IN A DISTRIBUITED COMPUTATION WE CAN HAVE 3 TYPE OF EVENTS

1. INTERNAL EVENT.

IT ONLY CHANGE THE STATE OF THE PROCESS THAT EXECUTES THE EVENT.

2. EXTERNAL EVENT.

2.1 SEND(m)

2.2 RECIEVE(m)

THESE TWO TYPE OF EVENTS ATTACH A MESSAGE INDICATED WITH THE LEETER "m"

CH.2 GLOBAL STATES, CUT AND CONSISTENCY

QUESTION 2: WHAT IS CONSISTENCY?

ANSWER: IN A D.S. WE HAVE CONSISTENCY WHEN THE MONITOR PROCESS p_0 (PROCESS THAT MAKE G.P.C.) RECIEVE COMPATIBLE STATES.

- HISTORY.

IN DISTRIBUITED SYSTEM WE HAVE TWO TYPE OF

HISTORY...

1. LOCAL HISTORY.

IS THE SET OF THE EVENTS EXECUTED BY A SPECIFIC PROCESS P_i

$h_i = e_i^1 e_i^2 e_i^3 \dots$ POTENCIALLY INFINITE...

↑ WHERE i IS REFERRED TO THE PROCESS P_i

2. GLOBAL HISTORY.

IS THE SET OF ALL THE EVENTS EXECUTED BY ALL THE PROCESSES IN THE DISTRIBUTED SYSTEM.

$H = h_1 \cup h_2 \cup h_3 \cup \dots \cup h_m$

$H = e_1^1 \dots e_1^{\dots} \cup e_2^1 \dots e_2^{\dots} \cup [\dots]$

- CAUSAL PRECEDENCE.

THE CAUSAL PRECEDENCE "$\rightarrow$" IS A RELATION ON THE EVENTS IN H

IF $e \rightarrow e' \Rightarrow e$ INFLUENCE e' AND FOLLOWS THE FOLLOWING PROPERTIES:

1. IF e, e' IS EVENT OF THE SAME PROCESS P_i AND e PRECEDES e' IN THE LOCAL HISTORY h_i THEN $e \rightarrow e'$.
2. IF e = SEND(m) AND e' = RECIEVE(m) THEN $e \rightarrow e'$
3. IF $e \rightarrow e'$ AND $e' \rightarrow e''$ THEN $e \rightarrow e''$.

- GLOBAL STATES, CUT, RUN

- TO PERFORM A G.P.E. THE MONITOR p_0 REQUIRES THE GLOBAL STATE OF THE SISTEM... THE GLOBAL STATE Σ IS DEFINED AS $\Sigma = \sigma_1, \sigma_2, \dots, \sigma_m$ WHERE σ_i IS THE LOCAL STATE OF P_i.
- A CUT IS A SUBSET IN THE GLOBAL HISTORY.

$C = h_1^{e_1} \cup \dots h_m^{e_m}$.

THE TUPLE $(e_1,\dots,e_m)$ INDICATE THE LAST EVENT TO CONSIDER IN THE LOCAL HISTORY.

- A RUN IS A TOTAL ORDER ON H, IF e PRECEDES e' IN LOCAL HISTORY h_i OF p_i THEN e PRECEDES e' IN R.
- A CUT IS CONSISTENT IF LEFT CLOSED OR RATHER: IF $\forall e, e' \in H$ AND $e' \in C$ AND $e \to e' \Rightarrow e \in C$.
- A RUN IS CONSISTENT IF $\forall e, e'$ $e \to e' \Rightarrow$ e APPEARS BEFORE e' IN R.

CH. 3 MONITORING A D.S.

TO MONITOR A D.S. WE CAN APPLY TWO TYPE OF PREDICATE... ACTIVE MONITOR OR PASSIVE MONITOR.

ACTIVE MONITOR.

IN SYSTEMS WITH ACTIVE MONITOR, THE MONITOR PROCESS p_0 ASKS CONTINOUSLY TO THE OTHER PROCESSES THEIR LOCAL STATE. WE CAN NOTICE SOME PROBLEMS IN THIS TYPE OF SYSTEM.

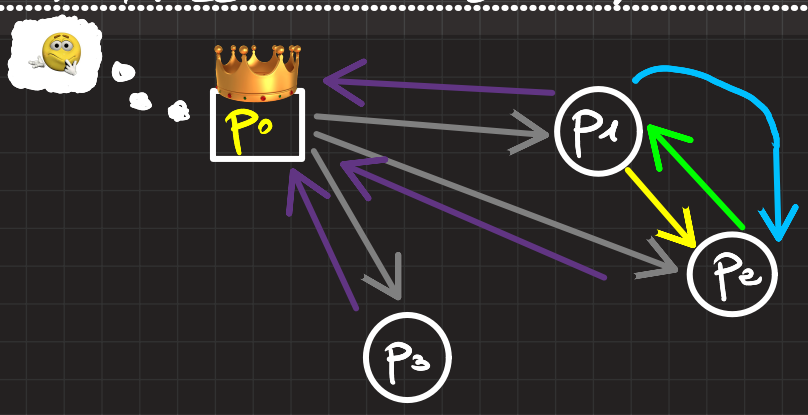
1. GLOBAL STATES THAT THE MONITOR RETRIEVE CAN BE INCONCOSISTENT AND ENCREASING THE FREQUENCY OF THE REQUESTS THE PROBLEM IS NOT FIXED.

2. GHOST DEADLOCK

WE ALSO COULD HAVE GHOST DEADLOCK...

BASIC EXAMPLE OF GHOST DEADLOCK.

EXAMPLE 1.



IN THE EXAMPLE WHE HAVE THE FOLLOWING SITUATION:

P_1 SEND TO P_2 A MESSAGE AND IT WAIT FOR A CONFIRMATION MESSAGE FROM P_2

P_2 SEND TO P_1 THE CONFIRMATION MESSAGE ... AND WAIT FOR A CONFIRMATION MESSAGE FROM P_1 ...

IF THE MONITOR P_0 SENDS THE STATE REQUEST IT CAN ARRIVES TO P_1 AND P_2 IN THE FOLLOWING MOMENT...

P_1 : WAITING FOR P_2 CONFIRMATION MESSAGE
P_2 : WAITING FOR P_1 CONFIRMATION MESSAGE

(THIS IS CAUSED BY THE IMPREVEDIBLE RETARDS ON THE COMMUNICATION LINE)

THE GLOBAL STATE RETRIVED IS AN INCONSISTENT STATE AND IT CONTAINS A GHOST DEADLOCK, OR RATHER A DEADLOCK THAT DOESN'T EXIST IN THE SYSTEM BUT EXIST IN G.P.E..

- PASSIVE MONITOR.

IN SYSTEM WITH PASSIVE MONITOR THE MONITOR PROCESS P_0 DOESN'T ASK TO THE OTHER THEIR STATE BUT THE PROCESSES IN D.S. SENDS TO THE MONITOR ONLY RELEVANT INFORMATIONS FOR THE G.P.E.

TO AVOID UNCONSISTENT CUT WE INTRODUCE FIFO RULE

- FIFO RULE

EACH PROCESS IN THE DISTRIBUITED SYSTEM HAS 2 LEVEL 1) DELIVERY LEVEL 2) APPLICATION

LEVEL.
ALL THE MESSAGES ARRIVE IN THE DELIVERY LEVEL AND DELIVERED TO THE APPLICATION LEVEL FOLLOWING THE ORDER DESCRIBED BY THE **FIFO RULE** OR RATHER:

A COMMUNICATION BETWEEN p_i AND p_j FOLLOW THE **FIFO DELIVERY RULE** IF $\forall m, m'$

$$SEND_i(m) \rightarrow SEND_i(m') \Rightarrow DELIVER_j(m) \rightarrow DELIVER_j(m')$$

where the pedice indicates the process that execute the event.

CH. 4 CLOCK

- UNTIL NOW WE HAVE ANALIZED DISTRIBUITED SYSTEM WITH NO CLOCK, NOW, SUPPOSE TO HAVE A SYSTEM WHERE THE MONITOR RECIEVES ALL THE MESSAGES EXCHANGED BETWEEN THE PROCESSES WITH AN ATTACHED **TIMESTAMP** AND P_0 STORE THESE MESSAGES IN A QUEUE AND DELIVERS THESE PERIODICALLY TO THE APPLICATION LEVEL.
THIS TYPE OF SYSTEMS SATISFIES **CLOCK CONDITION**

- **CLOCK CONDITION**
A FUNCTION T THAT ASSOCIATES A TIMESTAMP TO AN EVENT SATISFIES THE CLOCK CONDITION IF $e \rightarrow e' \Rightarrow T(e) < T(e')$.
BUT THE CLOCK CONDITION CAN BE SATISFIED BY OTHER TYPE OF SYSTEMS...

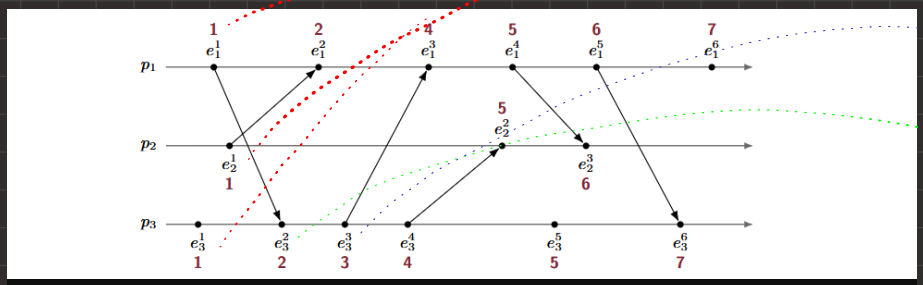
- LAND PORT LOGICAL CLOCKS.
SUPPOSE TO HAVE A COUNTER "LC" FOR EACH PROCESS p_i IN A DISTRIBUITED SYSTEM.
WHEN AN EVENT OCCOURRES THE LC ASSOCIATED TO THE EVENT FOLLOW THE FOLLOWING RULE:

$$LC(e_i) = \begin{cases} LC += 1 & \leftarrow \text{INTERNAL / SEND EVENT} \\ \max\{LC, TS(m)\} + 1 & \leftarrow \text{RECEIVE EVENT} \end{cases}$$

THIS TYPE OF SYSTEM ALSO SATISFIES THE **CLOCK CONDITION**...

TO BETTER COMPREND FOLLOW THE FOLLOWING EXAMPLE...

EXAMPLE 3.



- The first events e_1^1, e_1^2, and e_1^3 are either send events or internal events, so their clock values are 1. The requests sent in e_1^1 and e_1^2 carry a timestamp of 1.
- Event e_2^1 receives the request from process p_2 with timestamp 1: $\max\{1, 1\} + 1 = 2$. Similarly, e_3^3, which receives the request from p_1, gets a value of 2.
- Process p_3 sends a response, $LC(e_3^3) = 3$, and then sends a request to p_2, $LC(e_4^3) = 4$.
- Event e_3^1 receives the response from p_3 with timestamp 3: $\max\{2, 3\} + 1 = 4$. The clock of process p_1 jumps from 2 to 4 to catch up with p_3's past.
- Event e_2^2 receives the request from p_3 with timestamp 4: $\max\{1, 4\} + 1 = 5$.
- Process p_1 proceeds with the response to p_2, $LC(e_4^1) = 5$, the request to p_3, $LC(e_5^1) = 6$, and the internal event e_6^1, which has a value of 7.
- Event e_3^2 receives the response from p_1 with timestamp 5: $\max\{5, 5\} + 1 = 6$. Event e_5^3 is internal and has a value of 5; event e_6^3 receives the request sent in e_5^1 with timestamp 6: $\max\{5, 6\} + 1 = 7$. The clock of process p_3 jumps from 5 to 7 and never assumes the value 6.

- THESE TWO TYPE OF SYSTEM DON'T HAVE GAP DETECTION BECAUSE THEY CANNOT RECOGNIZE IF AN EVENT WITH LOWER TIMESTAMP THAN THE HIGHEST TIMESTAMP THAT THEY HAVE IN THEIR HAND CAN ARRIVE...

QUESTION 3: WHAT IS GAP DETECTION?

ANSWER: A SYSTEM HAS GAP DETECTION IF GIVEN $LC(e) < LC(e')$ CAN RECOGNIZE IF EXIST e'' WITH $LC(e) < LC(e'') < LC(e')$.

- HOW TO ADD GAP-DETECTION TO OUR SYSTEM?

WE CAN IMPLEMENT THIS IN A VERY SIMPLE WAY.

1) ADD FIFO CHANNELS IN EACH PROCESS OF THE D.S.

2) p_0 DELIVERS A MESSAGE ONLY IF IS STABLE.

- STABLE MESSAGE.

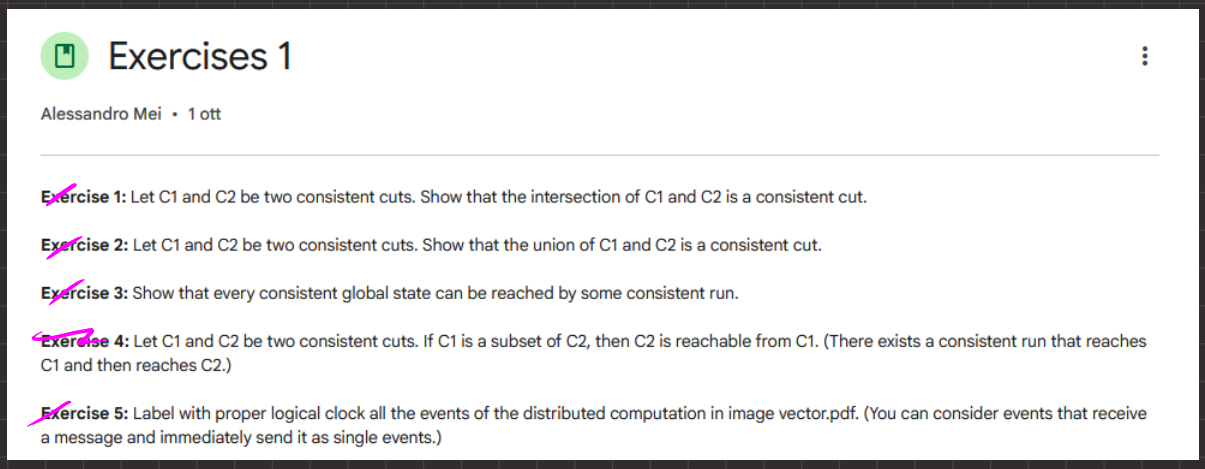
A MESSAGE m WITH TIMESTAMP $TS(m)$ IS STABLE FOR p_0 IF IT RECIEVE AT LEAST A MESSAGE m_1 WITH TIME STAMP $TS(m_1) > TS(m)$ FROM EACH PROCESS IN THE DISTRIBUITED SYSTEM.

WHY IT WORKS ???

BECAUSE IF p_0 RECIEVE A MESSAGE m WITH TIMESTAMP 4 FROM p_1, A MESSAGE m_1 WITH TIMESTAMP 5 FROM p_2 AND A MESSAGE m_{11} WITH TIMESTAMP 6 FROM p_3 IN A D.D. $\{p_0, p_1, p_2, p_3\}$, p_0 IS SURE TO NOT RECIEVE A MESSAGE WITH TIMESTAMP LOWER THAN 4 BECAUSE EACH PROCESS HAS A FIFO CHANNEL.

PROBLEM: A PROCESS CAN STOP SEND MESSAGES

SOLUTION: p_0 ASK TO STOPPED PROCESSES FOR CONFIRM MESSAGES, THESE RESPONSES WILL HAVE BIGGER TS THAN THE MESSAGES IN QUEUE.



Exercises 1

Alessandro Mei • 1 ott

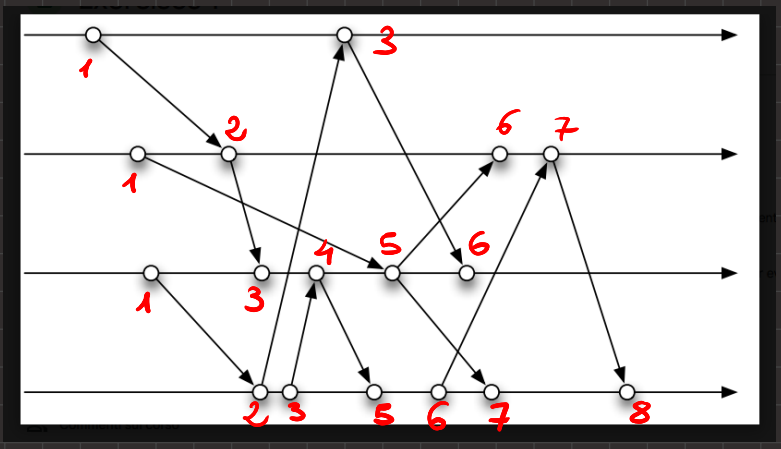
Exercise 1: Let C1 and C2 be two consistent cuts. Show that the intersection of C1 and C2 is a consistent cut.

Exercise 2: Let C1 and C2 be two consistent cuts. Show that the union of C1 and C2 is a consistent cut.

Exercise 3: Show that every consistent global state can be reached by some consistent run.

Exercise 4: Let C1 and C2 be two consistent cuts. If C1 is a subset of C2, then C2 is reachable from C1. (There exists a consistent run that reaches C1 and then reaches C2.)

Exercise 5: Label with proper logical clock all the events of the distributed computation in image vector.pdf. (You can consider events that receive a message and immediately send it as single events.)



vector.pdf

EXERCISE 1

LET e_1 AND e_2 CONSISTENT CUTS SHOW THAT THE INTERSECTION $(e_1 \cap e_2)$ IS A CONSISTEN CUT...

- A CUT IS CONSISTENT IF IS LEFT CLOSED, OR RATHER, IF $\forall e, e' \in H, e \in e \wedge e \to e' \Rightarrow e \in e$ WHERE "H" IS GLOBAL HISTORY AND "e" IS A CUT.

WE HAVE TO DEMONSTRATE THAT:

$\forall e, e' \in H, e' \in (e_1 \cap e_2), e \to e' \Rightarrow e \in (e_1 \cap e_2)$... AND THIS IS TRUE BECAUSE IF $e' \in (e_1 \cap e_2) \Rightarrow (e' \in e_1) \wedge (e' \in e_2)$

NOW, IF $e \to e'$ AND $e' \in (e_1 \cap e_2) \Rightarrow (e' \in e_1) \wedge (e' \in e_2) \Rightarrow$

$e \in e_1 \wedge e \in e_2 \Rightarrow e \in (e_1 \cap e_2)$ WHERE $e, e' \in H$.

EXERCISE 2

LET e_1 AND e_2 CONSISTENT CUTS SHOW THAT THE UNION $(e_1 \cup e_2)$ IS A CONSISTEN CUT...

- A CUT IS CONSISTENT IF...

WE HAVE TO DEMONSTRATE THAT:

$\forall e, e' \in H, e' \in (e_1 \cup e_2), e \to e' \Rightarrow e \in (e_1 \cup e_2)$... AND THIS IS TRUE BECAUSE IF $e' \in (e_1 \cup e_2) \Rightarrow (e' \in e_1) \vee (e' \in e_2)$

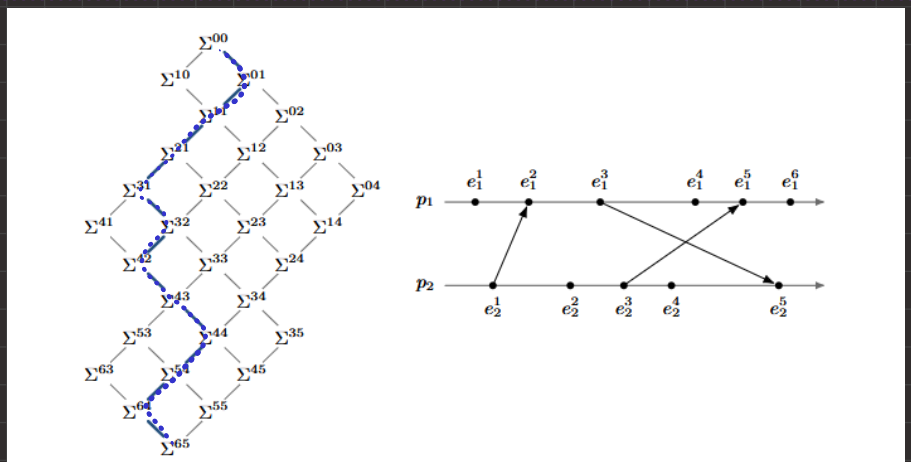
NOW, IF $e \to e'$ AND $e' \in (e_1 \cup e_2) \Rightarrow (e' \in e_1) \vee (e' \in e_2) \Rightarrow$

$e \in e_1 \vee e \in e_2 \Rightarrow e \in (e_1 \cup e_2)$ WHERE $e, e' \in H$.

EXERCISE 3

DEMONSTRATE THAT ALL THE CONSISTENT GLOBAL STATE CAN BE REACHED BY SOME CONSISTENT RUN...

- WE CAN REPRESENT A DISTRIBUTED COMPUTATION WITH THE LATTICE OF GLOBAL STATE, IN THIS REPRESENTATION WE HAVE STATES: $\Sigma^{00\ldots00}$, $\Sigma^{00\ldots01}$, $\Sigma^{00\ldots10}$ [...] AND THESE ARE ALL CONSISTENT STATES, THE NUMBERS ON TOP ARE THE NUMBER OF OPERATIONS EXECUTED BY THE i-TH PROCESS AND ARE m WHERE m IS THE NUMBER OF PROCESSES IN THE DISTRIBUITED SYSTEM
- A RUN IS A TOTAL ORDER ON A SUBSET OF THE HISTORY AND A RUN R IS CONSISTENT IF $\forall e, e' \in R$ IF $e \to e' \Rightarrow e$ APPEARS BEFORE e' IN R.
- IN A LATTICE OF GLOBAL STATES WE HAVE ALL THE POSSIBLE CONSISTENT STATES THAT A CERTAIN D.S. CAN REACH... IF WE MOVE FROM A STATE TO ANOTHER STATE THE NUMBERS ON THE STATES ARE DIFFERENT 'CAUSE SOME EVENT OCCOURRED AND THESE EVENT FORM A CONSISTENT RUN.



..... : $\Sigma^{00}\,\Sigma^{01}\,\Sigma^{11}\,\Sigma^{21}\,\Sigma^{31}\,\Sigma^{32}\,\Sigma^{42}\,\Sigma^{43}\,\Sigma^{44}\,\Sigma^{54}\,\Sigma^{64}\,\Sigma^{65}$

AND CORRESPONT TO

$e_2^1 e_1^1 e_1^2 e_1^3 e_2^2 e_1^4 e_2^3 e_2^4 e_1^5 e_1^6 e_2^5$.

THAT IS A CONSISTENT RUN

- WE CAN DEMONSTRATE IT BY INDUCTION...

BASE CASE :

WE CAN TAKE $\Sigma^{00...00}$, THAT IS THE INITIAL STATE THIS IS A CONSISTENT STATE AND THERE EXIST A RUN (NULL RUN) THAT REACH THIS STATE.

INDUCTIVE IP:

SUPPOSE VERIFIED UNTIL LEVEL $(m+1)$

INDUCTIVE CASE:

- NOW WE HAVE TO VERIFY IF IT IS REAL FOR LEVEL $(m+1)$
- WE HAVE SUPPOSED THAT UNTIL LEVEL $(m+1)$ WE HAVE CONSISTENT RUN FOR EACH STATE IN THE LATTICE, NOW TO GO TO A STATE IN LEVEL $(m+1)$ WE HAVE TO GENERATE AN EVENT AND WE HAVE 3 CASE OF EVENT

1) INTERNAL EVENT

   VERIFIED BECAUSE WILL EXIST A RUN WITH THIS EVENT

2) SEND //

3) RECIEVE //

∎

ESERCIZIO 4

LET c_1, c_2 CONSISTENT CUTS AND $c_1 \subseteq c_2$ THEN THERE EXIST A RUN THAT REACH c_1 AND THEN REACH c_2.

SUPPONING TO HAVE A CUT c_1 AND A CUT c_2

- IF $e' \in c_2 \wedge \forall e \in c_1 \; e \| e' \Rightarrow \exists R$ CONSISTENT RUN THAT REACH c_1 AND THEN c_2 BECAUSE IS ALWAYS

POSSIBLE TO ADD A NEW INDIPENDENT EVENT TO A RUN.

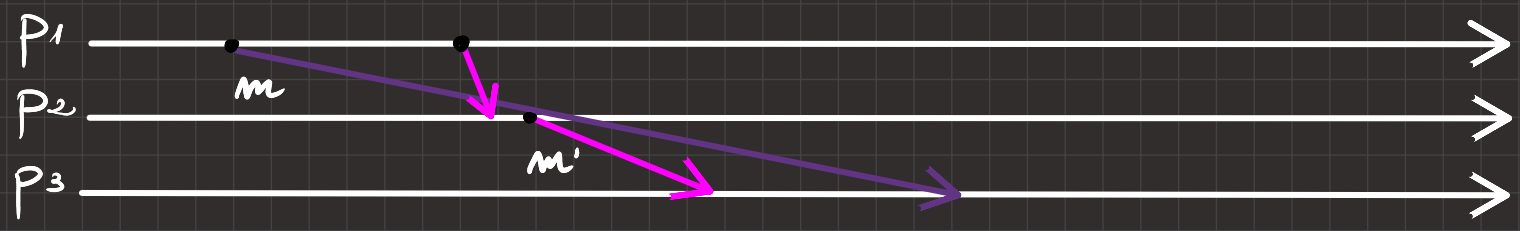
- IF $e' \in c_2 \wedge \exists e \in c_1 : e \to e' \Rightarrow \exists R$ CONSISTENT RUN THAT REACH c_1 AND THEN c_2 BECAUSE IS LEFT CLOSED

CH.5 CAUSAL DELIVERY AND VECTOR CLOCKS.

Carlo Re Rome NOV.6.2026

IN THE MODELS ANALIZED UNTIL NOW WE CAN NOTICE A VERY STRONG PROBLEM

EXAMPLE 5.



WE CAN NOTICED THAT $SEND_i(m) \to SEND_k(m_1)$ BUT $DELIVER_j(m') \to DELIVER_j(m)$

BECAUSE THE SYSTEM ANALIZED CAN HANDLE VERY WELL MESSAGES FROM ONE SOURCHE BUT IT HAS PROBLEMS WITH MORE...

- CAUSAL DELIVERY RULE

A SYSTEM SUPPORTS CAUSAL DELIVERY RULE IF $\forall m, m'$ $SEND_i(m) \to SEND_j(m') \Rightarrow$ $DELIVER_k(m) \to DELIVER_k(m')$.

IF $i = j$ CAUSAL DELIVERY BECOME FIFO DELIVERY RULE

CAUSAL DELIVERY RULE IS A PERFECTIONATED

FIFO DELIVERY RULE.

- **STRONG CLOCK CONDITION**

A SYSTEM RESPECT THE STRONG CLOCK CONDITION IF GIVEN A FUNCTION T THAT ASSIGN A VALUE TO EACH EVENT $\forall e, e' \in H$ IF $e \rightarrow e' \Leftrightarrow T(e) < T(e')$.

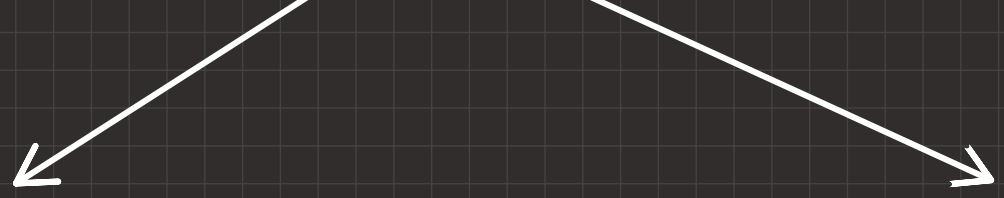
QUESTION 4. HOW TO IMPLEMENT THIS TWO IMPORTANT RULES IN A SYSTEM ??

ANSWER. VECTOR CLOCKS

- VECTOR CLOCKS.

PHASE 1) EACH PROCESS IN THE DISTRIBUTED SYSTEM INITIALIZE A VECTOR VC LONG IN CELLS AS THE SAME NUMBER OF PROCESSES IN THE D.S.

PHASE 2) IF AN EVENT OCCOURRES IN PROCESS p_i



INTERNAL / SEND

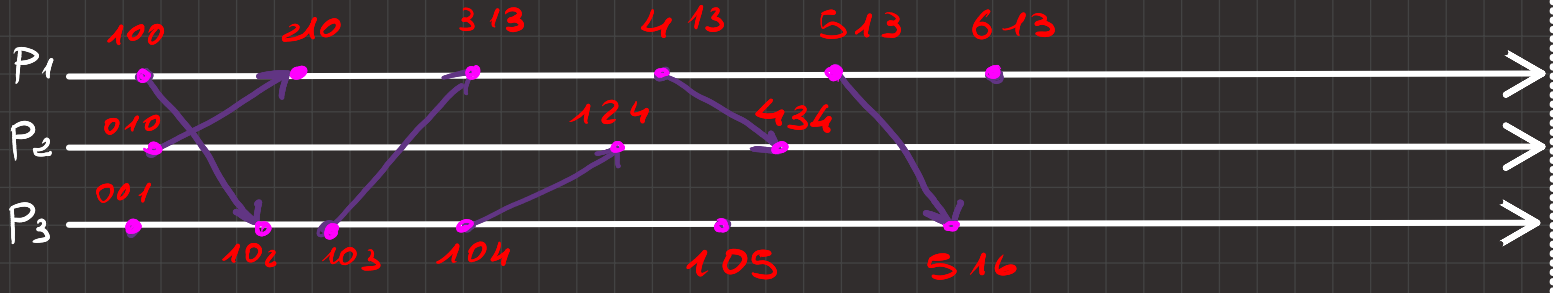
- $VC[i] += 1$

RECEIVING

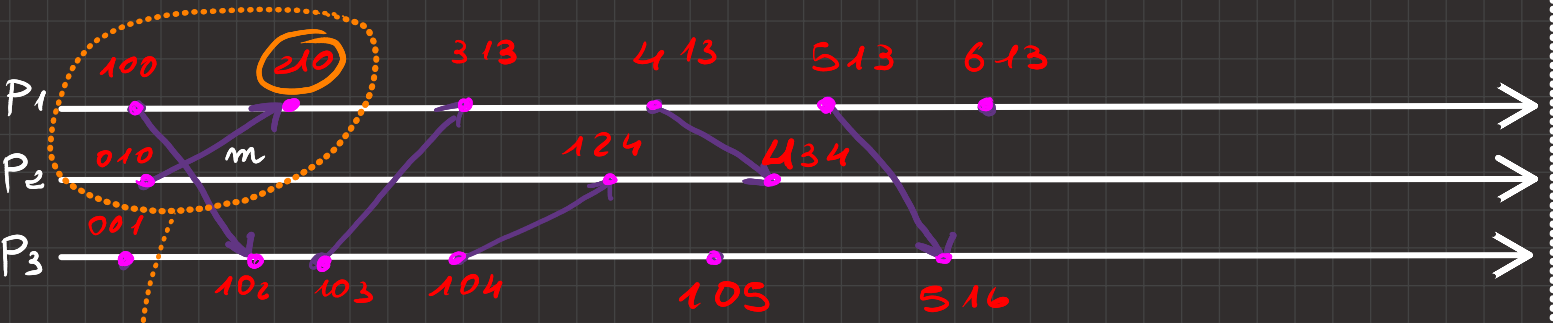
1. 1) $\forall_J \; VC[J] = max\{VC[J], TS(m)\}$
   2) $VC[i] += 1$
   (where J is a process.)
2. TS(m) is the VECTOR CLOCK ATTACHED TO THE MESSAGE m
3. EACH TIME THAT A PROCESS SENDS A MESSAGE IT ATTACH ITS VECTOR CLOCK TO THE MESSAGE (TS(m)).

EXAMPLE 3.

EXAMPLE OF A COMPUTATION



MORE IN DETAILS ...



210 BECAUSE
$TS(m) = 010$ WHILE
VC OF $p_1 = 100$

1) $VC[0] = \max\{1,0\} = 0$

$VC[1] = \max\{1,1\} = 0$

$VC[2] = \max\{0,0\} = 0$

2) $VC[i] += 1$

$VC = (2,1,0)$

BEFORE TO GO AHEAD WE HAVE TO TALK ABOUT

- CAUSAL HISTORY

THE CAUSAL HISTORY OF AN EVENT $\mathcal{O}(e)$ IS DEFINED AS $\mathcal{O}(e) = \{e' : e' \to e\} \cup \{e\}$

- COMPARE TWO CAUSAL VECTORS

TWO CAUSAL VECTOR ARE UNCOPARABLE IF

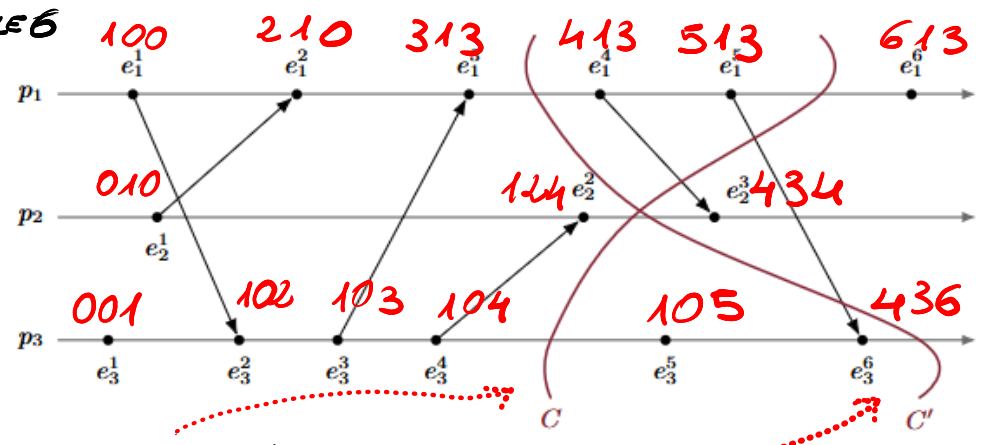
$!(CV(e) < CV(e')) \wedge !(CV(e) \geq CV(e'))$ ...

WHILE IF TWO CAUSAL VECTOR ARE COMPARABLE

$e \to e' \iff VC(e) < VC(e')$ (the system respect the clock condition)

- HOW TO KNOW IF A CUT IS CONSISTENT?

EXAMPLE6



$e = (5,2,4)$ $e' = (3,2,6)$

A CUT IS CONSINSTENT IF:

$VC(e_j^{e_j}) \leq (c_1, \dots, c_m)$ for each process p_j

WHERE $e_j^{e_j}$ IS THE LAST EVENT PERFORMED BY p_j IN THE CUT AND $(c_1, \dots, c_m)$ IS THE NUMBER OF EVENTS PERFORMED BY EACH PROCESS IN THE CUT

p_1 PERFORMED c_1 EVENTS IN THE CUT

p_2 PERFORMED c_2 EVENTS IN THE CUT

[...]

p_m PERFORMED c_m EVENTS IN THE CUT

THIS METHOD WORKS WELL BECAUSE IF A VC IS GRATHER THAN THE TUPLE, THIS MEANS THAT IT KNOW MORE EVENTS THAN THE EVENTS IN THE CUT AND IT IS COUNTING SOME EVENTS OUTSIDE THE CUT...

- SOMETHING MISSING IN THE MIDDLE?

THE SYSTEM USES THE WEAK GAP DETECTION.

$VC(e)[k]$ MEANS THAT e KNOW THE e EVENTS PERFORMED BY P_k ... IF

$VC(e)[k] < VC(e')[k] \Rightarrow \exists$ EVENT f PERFORMED BY p_k WHERE $f \not\to e \wedge f \to e'$.

THIS IS WEAK GAP DETECTION BECAUSE

"e DON'T KNOW ABOUT f" IS NOT "f CAME AFTER e"

FOR EXAMPLE: $VC(e_1^2) = (2,1,0)$ AND $VC(e_2^3) = (4,3,4)$

- CAUSAL DELIVERY WITH VECTOR CLOCK.

WE HAVE 3 ACTORS

- p_1, p_2. TWO COMMON PROCESSES
- p_0. THE MONITOR PROCESS

EACH MESSAGE m TO p_0 HAS $TS(m)$ ATTACHED

IF THE SENDER IS p_J:

* $TS(m)[J]$: INDICATE THE NUMBER OF NOTIFICATIONS SENDED BY p_J
* $TS(m)[K]$: INDICATE HOW MUCH NOTIFICATIONS p_J KNOW ABOUT SENDED BY p_K IN THE MOMENT WHEN p_J SEND THE CURRENT.

THE MONITOR HAS A VECTOR, D, WHERE $D[k]$ INDICATE HOW MUCH NOTIFICATIONS BY p_K THE MONITOR HAS ALREDY DELIVERED.

- DR3 RULE

AGENZIA DOGANE MONOPOLI

WHEN A NOTIFICATION m ARRIVE FROM p_J TO p_0, BEFORE THE DELIVERING p_0 FOLLOW THIS TWO CHECKS...

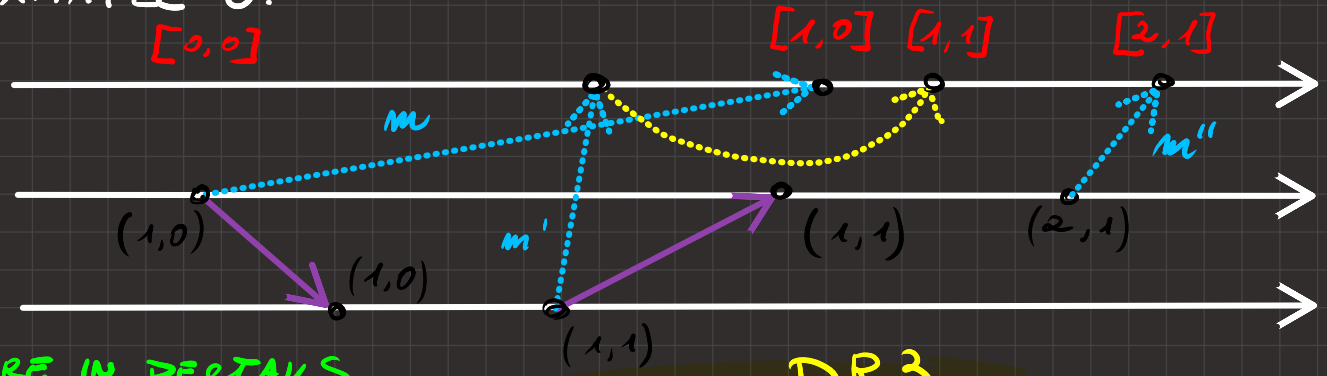
1. $D[J] = TS(m)[J] - 1$
2. $D[k] \geq TS(m)[k] \quad \forall k \neq J$

FOLLOWING THIS TWO RULES THE MONITOR IS SURE TO DELIVER THE MESSAGE IN CORRECT

ORDER.

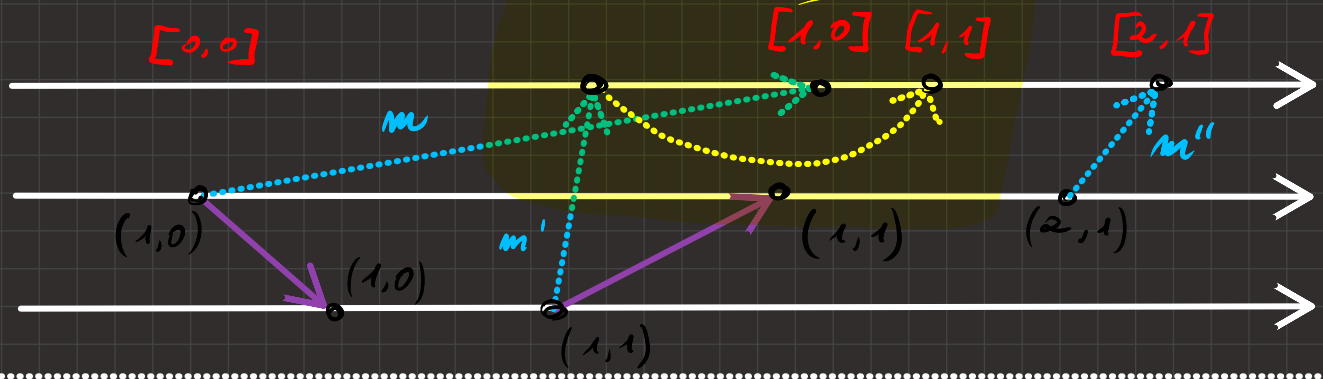
IF THE ANSWER TO THIS TWO POINTS IS NO THE MESSAGE IS PARKED IN A WAIT QUEUE

EXAMPLE 6.



MORE IN DETAILS

DR3



- THE MESSAGE m ARRIVES AFTER m' TO P_0 BUT $m \to m'$

$TS(m') = (1,1)$, $TS(m) = (1,0)$

THE CONDITION 2. IS NOT VERIFIED BECAUSE

$D = [0,0]$ AND $TS(m') = (1,1)$

$D[k] \geq TS(m)[k] \quad \forall k \neq j$ BUT $0 < 1$.

m' IS PARKED IN THE WAITING QUEUE AND DELIVERED WHEN THE CONDITIONS 1. AND 2. WILL B SATISFIED.

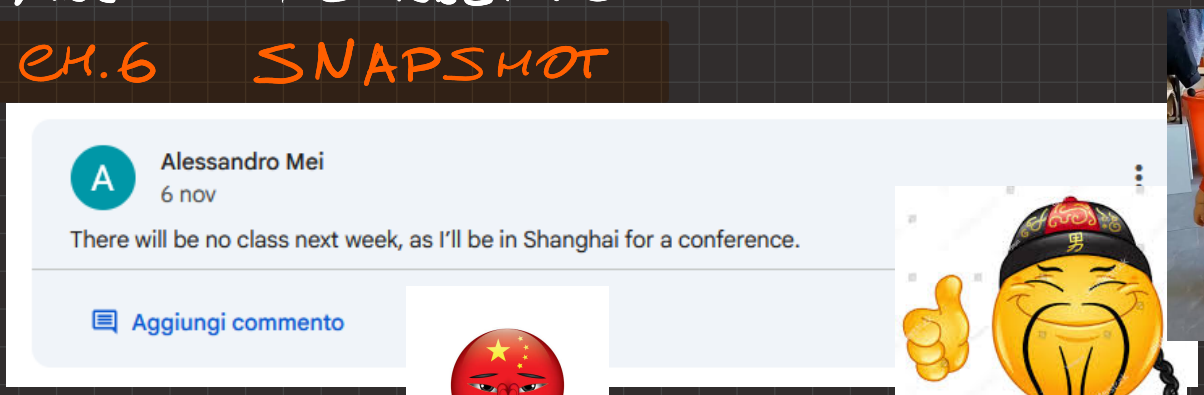
- HIDDEN CHANNELS

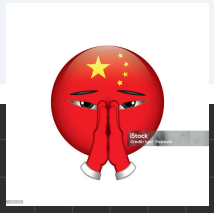
THIS WORKS VERY WELL IN SYSTEMS THAT DOESN'T COMPREEND PHYSICAL SYSTEMS.

- FOR EXAMPLE IF P_1 DETECT A BROCKEN TUBE AND P_2 DETECT LOW PRESSURE CAUSATED BY THE BROCKEN TUBE...

MESSAGES CAN ARRIVE INVERTED IN ORDER AND THE MONITOR CAN HAVE A DISTORTED PERCEPTION BECAUSE FOR IT THE MESSAGES ARE CONCURRENTS.

CH.6 SNAPSHOT





Corso Da Roma NOV. 9. 2026

IN THE CH2 THE MONITOR ASKED TO PROCESSES THEIR STATE, BUT THE GLOBAL STATE THAT COME OUT IS INCONSISTENT AND IT REQUIRED TO STOP COMPUTATION TO ASK AND RECEIVE STATES FROM PROCESSES...

- CANDY-LAMPORT
  - WHAT IS A SNAPSHOT ?

    A SNAPSHOT IS THE GLOBAL STATE AND MESSAGES ON CHANNELS OF A SYSTEM
  - WHAT IS CANDY LANDPORT ?
  - THAT IS AN ALGORITHM TO BUILD A CONSISTENT SNAPSHOT WITHOUT STOPPING THE COMUTATION.
  - IT USES A SPECIAL MESSAGE, THE MARKER MESSAGE. IT USES FIFO CHANNELS.

THE ALGORITHM:

1. Po SEND A MARKER TO HIMSELF

- WHEN p_i RECEIVE A MARKER FROM p_s

1. IF IT HAS NEVER RECEIVED A MARKER BEFORE
   1.1. SAVE ITS STATE
   1.2 SEND BROADCAST MARKERS
   1.3 MARK AS EMPTY THE COMMUNICATION CHANNEL $p_i - p_s$ AND STARTS LISTENING THE OTHER CHANNELS.
2. IF IT HAS ALREDY RECEIVED A MARKER BEFORE
   2.1 IT STOP COMMUNICATION $p_i - p_s$
3. IF IT HAS ALREDY RECEIVED MARKER FROM EVERYONE PROCESS IN THE D.S. IT SEND STATE AND MESSAGE QUEUE TO THE MONITOR.

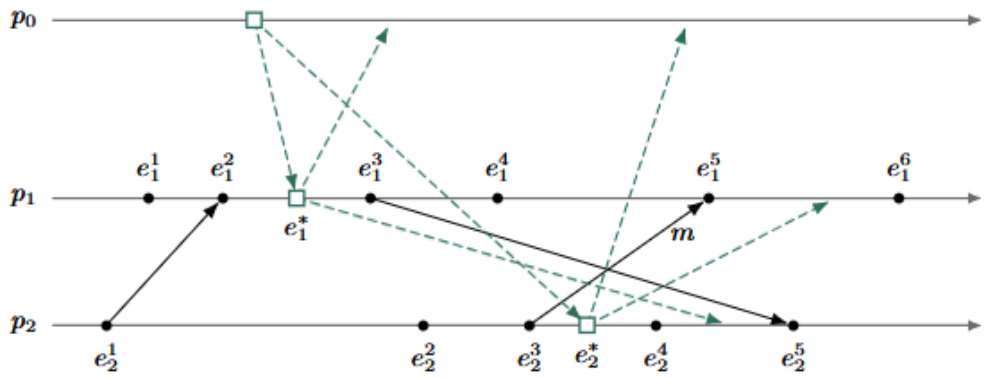
- MESSAGE COUNTING AND CONSISTENCY

MESSAGES CAN BE

1. SENDED BEFORE SENDER PHOTO AND RECEIVED BEFORE RECIPIENT PHOTO
   - COUNTED IN RECIPIENT LOCAL STATE
2. SENDED BEFORE SENDER PHOTO AND RECEIVED AFTER RECIPIENT PHOTO
   - COUNTED IN CHANNEL
3. SENDED AFTER SENDER PHOTO
   - NOT IN SNAPSHOT

- THAT GUARANTEES THAT THE SNAPSHOT IS CONSISTENT BECAUSE IS IMPOSSIBLE TO FIND A MESSAGE RECEIVED WITHOUT THE RELATIVE SEND EVENT.
- THAT SNAPSHOT CAN DESCRIBE A SITUATION THAT NEVER APPENED BUT IT IS CONSISTENT.

EXAMPLE. 7



- The message from e_2^1 arrives at p_1 before p_1's snapshot, like A in the second execution: it is already counted in σ_1^2;
- m leaves p_2 before p_2's snapshot, and arrives at p_1 after p_1's snapshot and before p_2's marker, like A in the first execution: it enters the list, and $\chi_{2,1} = \{m\}$;
- The message from e_1^3 leaves p_1 after p_1's snapshot, like B: it arrives at p_2 after p_1's marker, when the list is already closed, and $\chi_{1,2} = \emptyset$.

The snapshot is the global state Σ^{23} (two events of p_1, three of p_2), with m in transit from p_2 to p_1.

Exercises 2

Alessandro Mei • 2 ott

Exercise 6: Show that the history of an event is a consistent cut.

Exercise 7: Show that the Chandy-Lamport Snapshot Protocol builds a consistent global state.

Exercise 8: Show that the Chandy-Lamport Snapshot Protocol can build a global state that never happened.

Exercise 9: What good is a distributed snapshot when the system was never in the state represented by the distributed snapshot? Give an application of distributed snapshots.

Exercise 10: Consider a distributed system where every node has its physical clock and all physical clocks are perfectly synchronized. Give an algorithm to record global state assuming the communication network is reliable. (Note that your algorithm should be simpler than the Chandy–Lamport algorithm.)

Exercise 11: What modifications should be done to the Chandy–Lamport snapshot protocol so that it records a strongly consistent snapshot (i.e., all channel states are recorded empty).

Exercise 12: Show that, if channels are not FIFO, then Chandy–Lamport snapshot algorithm does not work.

Exercise 13: Let S0 be the global state when the Chandy-Lamport snapshot protocol starts, S be the global state built by the protocol, and S1 be the global state when the protocol ends. Show that S is reachable from S0 and that S1 is reachable from S. Remember that S might not have happened. (In other words, there is a consistent run that goes through S0, S1 and S.)

EXERCISE 6:

SHOW THAT THE HISTORY OF AN EVENT IS A CONSISTENT CUT

mmmh ...

1 A CUT IS A SUBSET OF THE GLOBAL HISTORY

2 A CONSISTENT CUT C IS A CUT WHERE IF $e, e' \in H, e' \in C \Rightarrow e \rightarrow e'$

3 THE HISTORY OF AN EVENT e IS DEFINED AS:

$$\phi e = \{e' : e' \to e\} \cup \{e\}.$$

LET'S SUPPOSE THAT (ϕe) IS NOT A CONSISTENT CUT...

4 IF $e' \in (\phi e)$

$e' \to e$

SUPPOSE THAT:

$\exists\, e'' \notin (\phi e)$ $e'' \to e'$ (BUT) FOR DEFINITION OF (ϕe)

$e'' \in (\phi e)$

THAN, THE CUT IS CONSISTENT

EXERCISE 7

DEMONSTRATE THAT CANDY-LANDPORT BUILDS CONSISTENT GLOBAL STATE...

- WE CAN DIVIDE CANDY-LANDPORT IN TWO STAGES
  1. PREREGISTRATION
  2. POSTREGISTRATION
- WHEN A PROCESS TAKE THE PHOTO TO ITS STATE IT IS THE POSTREGISTRATION PHASE, BEFORE THAT, IT IS IN PREREGISTRATION PHASE.
- WE CAN FIND 3 TYPE OF MESSAGE SITUATIONS
  1. SENDER SEND A MESSAGE BEFORE THE REGISTRATION AND THE RECIPIER RECEIVE THE MESSAGE BEFORE REGISTRATION

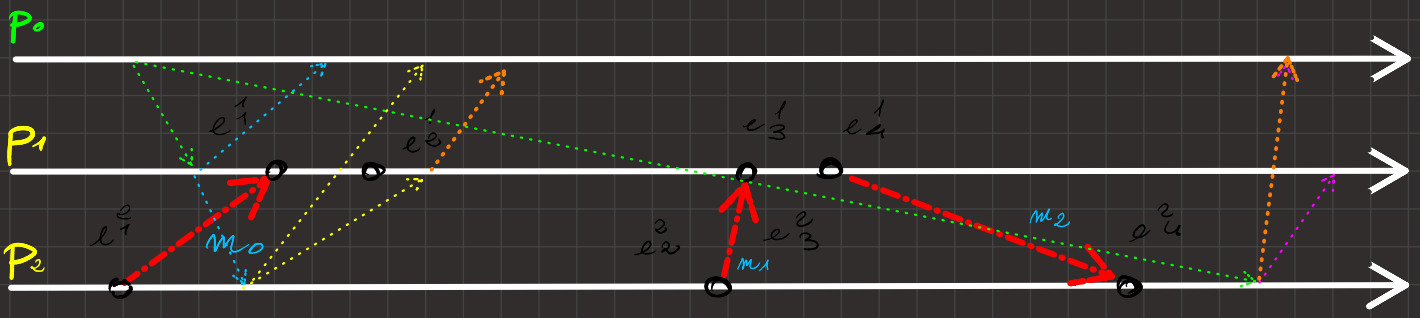
2. SENDER SEND A MESSAGE BEFORE THE REGISTRATION AND THE RECIPIER AFTER THE MESSAGE BEFORE REGISTRATION
3. SENDER SEND A MESSAGE AFTER THE REGISTRATION

- IN CASE 1. THE SEND ACTION IS REGISTERED IN THE LOCAL STATE OF THE SENDER AND THE RECEIVE ACTION IS REGISTERED IN THE LOCAL STATE OF THE RECIPIER.
- IN CASE 2. THE MESSAGE IS CONSIDERED ON CHANNEL
- IN CASE 3. THE MESSAGE IS NOT IN THE SNAPSHOT.
- WITH THIS ARCHITECTURE IS IMPOSSIBLE TO FIND EVENTS WITHOUT ITS RELATIVE CAUSE IN THE BUILT STATE.

☒

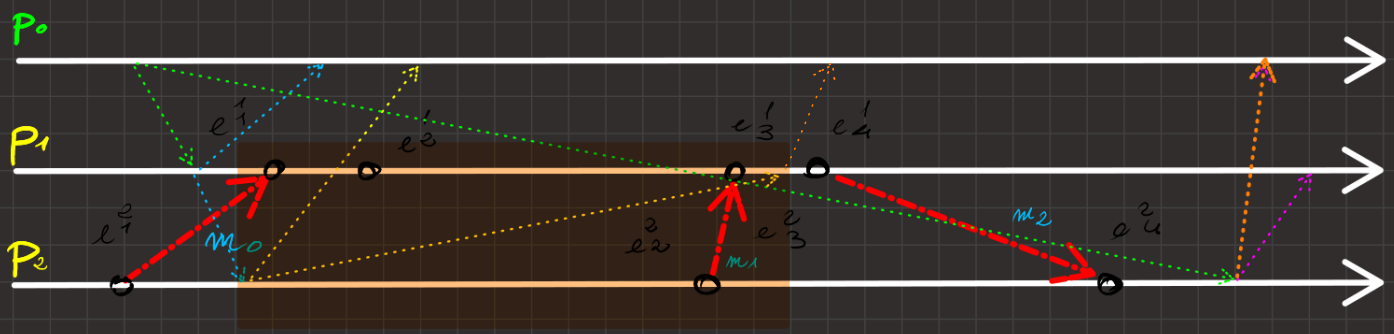
EXERCISE 8

DEMONSTRATE THAT CANDY-LANDPORT CAN BUILD GLOBAL STATES THAT NEVER HAPPENED...



$$SNAPSHOT = \{\{e_1^2\}, \{m_0\}\}$$

IS A CHRONOLOGICAL IMPOSSIBLE GLOBAL STATE.



SITUATION LIKE THIS ARE IMPOSSIBLE BECAUSE KANDY LANDPORT USES FIFO CHANNELS.

$SEND_i(m_0) \rightarrow SEND_i(m_1) \Rightarrow DELIVER_j(m_0) \rightarrow DELIVER_j(m_1)$

ESERCIZIO 9

GIVE AN EXAMPLE OF APPLICATION OF DISTRIBUITED SNAPSHOTS.

- FOR EXAMPLE IF YOU WANT TO VERIFY IF A PROCESS IS NOT CRASHED
- ANY STABLE PROPERTIES

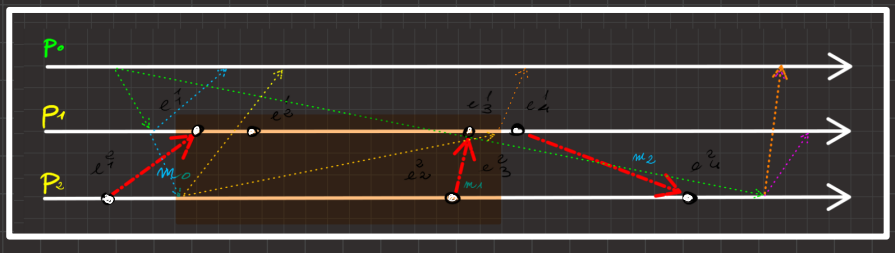
ESERCIZIO 10

1. P_0 SENDS "START" MESSAGE TO ALL THE PROCESSES
2. P_i RECIEVE "START", SAVE ITS STATE AND LISTEN THE IN CHANNELS WHILE t IS LESS THAN A T
3. P_i SENDS STATE AN STATE OF CHANNELS TO P_0.

ESERCIZIO 11

STRONGLY CONSISTENT ?

ESERCIZIO 12



IN THIS CASE WE HAVE m_1 IN THE SNAPSHOT BUT THE EXPEDITION OF m_1 HAPPENED A LOT OF TIME AFTER THE MOMENT CAPTURED IN THE SNAPSHOT.

Exercise 13: Let S0 be the global state when the Chandy-Lamport snapshot protocol starts, S be the global state built by the protocol, and S1 be the global state when the protocol ends. Show that S is reachable from S0 and that S1 is reachable from S. Remember that S might not have happened. (In other words, there is a consistent run that goes through S0, S1 and S.)

WE HAVE TO DEMONSTRATE THAT:

$\Sigma_{S0} \rightarrow \Sigma_S \rightarrow \Sigma_{S1}$

- WE KNOW THAT GIVEN $c_1 \subseteq c_2$ WHERE c_1 AND c_2 ARE TWO CONSISTENT CUT, EXIST A PATH $c_1 \rightarrow c_2$ ON THE LATTICE OF STATES...
- $\Sigma_{S0} \subseteq \Sigma_S \subseteq \Sigma_{S1}$ AND ALL THIS CUT ARE CONSISTENT

THEN

EXIST A PATH $\Sigma_{S0} \rightarrow \Sigma_S \rightarrow \Sigma_{S1}$. ☑

CH. 7. FAILURE, ATOMIC COMMIT AND TPC

UNTIL NOW SYSTEMS DOESN'T CRASH AT ALL BUT REAL SYSTEMS CRASH AND WE HAVE TO HANDLE THESE PROBLEMS...

DISTRIBUITED TRANSACTIONS

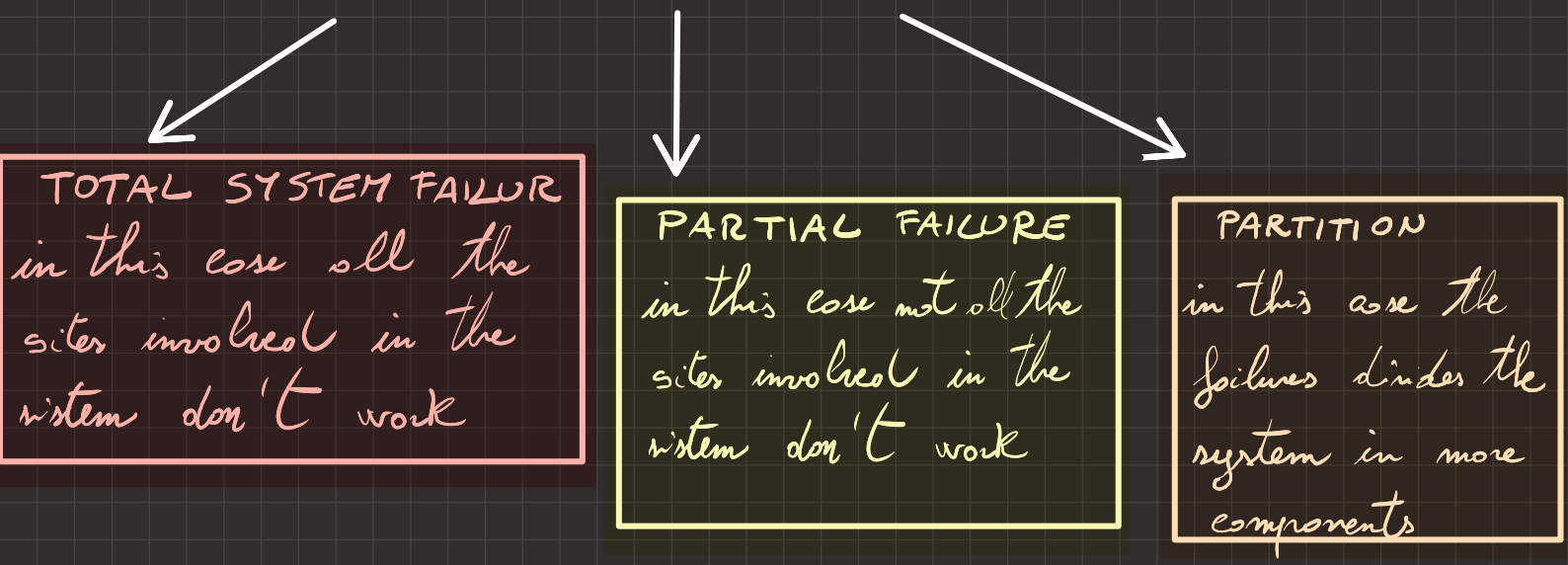
Carlo De Rome NOV. 10. 2026

- IN A DISTRIBUTED SYSTEM WE CAN IDENTIFY SOME PARTS

1. SITES = PROCESSES
2. HOME SITE = A SYSTEM THAT HANDLE THE OPERATIONS OF ONE SPECIFIC SITE
3. TRANSACTION MANAGER = A SYSTEM IN THE HOME SITE THAT HANDLE WRITING AND LECTURES ON THE SITE.

- WHEN A SITE WANT TO DO A DISTRIBUTED TRANSACTION ALL THE OTHER SITES HAVE TO ACCEPT THE TRANSACTION OTHERWISE THE TRANSACTION TERMINATES WITH AN ABORT.

- THE ABORTION OF A TRANSACTION IS NOT THE MOST IMPORTANT PROBLEM IS THE CONSISTENT TERMINATION... WE WANT THAT ALL THE SITES DO THE SAME OPERATION FOR THE TRANSACTION ABORT OR COMMIT
- A SYSTEM FAILUR COULD BE



- THE SITES WORKS FOLLOWING THE FAIL STOP PROTOCOLL, A SITE CAN ONLY ⇄ WORK / DON'T WORK BUT IT CAN'T WORK PARTIALLY.
- BEFORE DOING A DISTRIBUTE TRANSATION A COORDINATOR SITE HAS TO CHECK IF 2. RULES ARE RESPECTED BY ALL THE PARTECIPANT SITES.
  1. THERE ARE N'T NOT-CONFIRMED TRANSACTIONS
  2. THERE AREN'T DATA ON THE VOLATILE MEMORY OF THE SITE.
- ALL THE SITES HAVE VOLATICE MEMORY (DT LOG) WHERE THEY COULD SAVE DATA USEFULL FOR RECOVERY.
- WHEN A TRANSACTION ARRIVE TO THE SITE, THE SITE CAN ANSWER WITH YES OR NO... IF EVEN JUST A SITE ANSWER NO THE COMMON DECISION IS ABORT...

NO: 1
SI: 250
RISULTATO: SE PISCIA.

- ATOMIC COMMIT PROTOCOL

EACH SITE CAN ANSWER WITH YES OR NO ... IF EVEN JUST A SITE ANSWER NO THE COMMON DECISION IS ABORT...

AN ATOMIC COMMIT PROTOCOL IS AN ALGORITHM THAT MAKE A DECISION FOLLOWING THESE 5 RULES:

1. ALL THE PROCESSES THAT REACH A DECISION REACH THE SAME
2. A PROCESS CAN'T CHANGE ITS DECISION
3. THE COMMIT IS REACHED IF ALL THE SITES VOTED YES
4. IF THERE AREN'T FAILURES AND ALL THE SITES VOTED YES, THE DECISION IS COMMIT
5. IN EACH EXECUTION IF SOME FAILURES ARE SOLVED AND THEY DON'T OCCURS FOR A LONG TIME SOONER OR LATER THE SISTEM WILL TAKES A DECISION.

- THE PROCESSES COULD BE IN AN UNCERTAIN PERIOD AND IT COULD CRASH IN THIS UNCERTAIN PERIOD...

I SCENARIO : IT CANNOT COMUNICATE WITH OTHER PROCESSES AND IT CANNOT RETRIVE INFORMATIONS BECAUSE IT IS BLOCKED IN A COMPONENT OF ONE PARTITION

II SCENARIO : IT CAN COMMUNICATE WITH OTHER PROCESSES AND RETRIVE INFORMATIONS.

- TIMEOUTS

TO CHECK IF A SITE IS CRASHED WE CAN USE TIMEOUTS

BUT

IS DIFFICULT TO CHOSE A CORRECT TIME FOR THE TIMEOUT AND SOMETIMES IT COULD SET AS TIMEOUTED GOOD SITES.

TWO LIMITS OF ATOMIC COMMIT PROTOCOL
- IF FAILURES ARE POSSIBLE EACH ACP CAN BLOCK PROCESSES
- ACPs CAN'T GUARANTEE INDEPENDENT RECOVERY

• TWO-PHASE COMMIT PROTOCOL

2 THINGS BEFORE STARTING:

Yes: IS THE CHOICE OF A PROCESS

YES: IS A MESSAGE

yes: IS A DTLOG LOG

: FAMOUS MEME 2021-2023

• THE PROTOCOL

1. THE COORDINATOR SENDS TO ALL THE PARTECIPANTS A MESSAGE "VOTE REQ"
2. EACH PARTECIPANT CAN ANSWER WITH "YES" OR "NO" IF IT VOTE "NO" IT DECIDE Abort AND IT STOPS.
3. THE COORDINATOR PEERS ALL THE VOTES AND DECIDES commit AND SENDS COMMIT MESSAGE OR DECIDES Abort AND SENDS ABORT MESSAGE TO THE COORDINATOR.
4. EACH PARTECIPANTS WAIT FOR COMMIT OR ABORT MESSAGE.

• TERMINATION PROTOCOL

- THE PROCESSES COULD BE IN UNCERTAIN STATE. THE SYSTEM REQUIRE A SOLUTION TO AVOID INFINITE EXPECTATIONS.
- EACH SITE p HAS A TIMEOUT, IF IT EXPIRES THE SITE RUNS THE TERMINATION PROTOCOL:

1. p THAT IS THE INIZIATOR SENDS TO THE OTHERS (RESPONSERS) DECISION-REQ, IF IT DOESN'T RECIVE A RESPONSE BEFORE THE END OF THE SECOND TIMEOUT IT RE-TRY THE TERMINATION PROTOCOL.
2. THE RESPONSERS RECEIVE THE REQUEST AND:
   - 2.1 IF A RESPONSER DECIDED Abort OR commit IT SENDS ABORT OR COMMIT.
   - 2.2 IF IT DOESN'T ALREDY DECIDED, IT SEND ABORT MESSAGE
   - 2.3 IT VOTED YES BUT IT NOT ALREDY DECIDED, IT DON'T SEND N

• RECOVERY, DT LOG, GARBAGE COLLECTOR

- BEFORE EXECUTING ANY ACTION THE SITE WRITE THE ACTION IN ITS DTLOG, FOR EXAMPLE BEFORE VOTING YES TO A VOTE-REQ THE SITE WRITE "YES" ON THE DT LOG, THIS CAN BE ERASE IF ALL THE DATAS OF THE TRANSACTION ARE IN STABLE MEMORY AND IF THE TRANSACTION IS ENDED WITH ABORT OR COMMIT.
- IF A SITE CRASH IT CAN EASLY RETURN AT THE STATE BEFORE THE CRASH FOLLOWING THIS PROTOCOL.

RECOVERY PROTOCOL:

1. IF IT CRASH BEFORE SEND A MESSAGE OR WRITING ON THE DT LOG IT DECIDE ABORT
2. IF IT DECIDED SOMETHING CAN EASLY RESTORE ITS STATE
3. IF IT WAS IN UNCERTAIN STATE IT STARTS TERMINATION PROTOCOL.

• 2PC EVALUATION

- ROUND

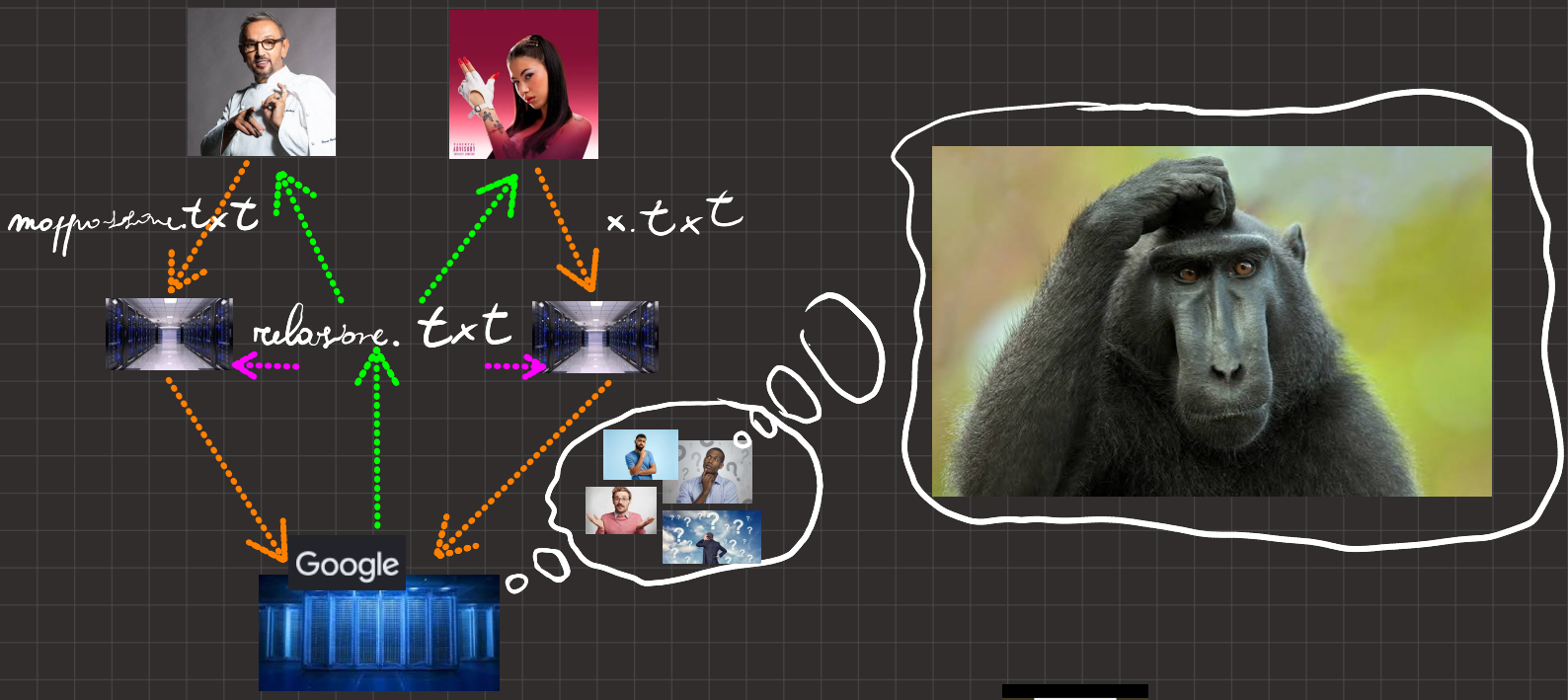
TIME COMPLEXITY IS MEASURED IN ROUNDS.
A ROUND IS THE MAX TIME THAT A MESSAGE USE TO ARRIVE.

* TIME COMPLEXITY: WITHOUT CRASHES IT REQUIRES 3 ROUNDS. THE TERMINATION PROTOCOL ADDS 2 ROUNDS.
* SPACE COMPLEXITY: WITHOUT CRASHES IT USES $3m$ MESSAGES (WHERE m IS THE NUMBER OF PARTECIPANTS) IF ALL THE PARTECIPANTS GOES IN TIMEOUT THE MESSAGES BECOME $m(3m+1)/2$.

* IT IS RESILIENT, BECAUSE IT TOLLERATES ALL TYPE OF FAILURES AND BLOCKS
* SITES MIGHT BLOCK IN UNCERTAIN PHASE.

CH. 8. CONSENSUS AND PAXOS.

- PAXOS IS A PROTOCOL USED BY GOOGLE TO MODIFY FILES CORRECTLY...
- FOR EXAMPLE IF ANNA AND BRUNO HAVE A SHARED DIRECTORY AND IN THE SAME MOMENT ANNA RENAME THE FILE relazione.txt in x.txt AND BRUNO IN moppastone.txt THEY WANT THAT THE FILE IS ONE AND WITH ONE TITLE (DON'T CARE ABOUT WHICH ONE).



- TO DO THAT WE USE PAXOS
  IN PAXOS WE HAVE 3 ROLES
  - proposer: SITE THAT PROPOSE A VALUE.
  - acceptor: SITE THAT VOTE A VALUE.
  - learner: SITE THAT LEARN A VALUE.
- CONSENSUS

A PROTOCOL SOLVE CONSENSUS IF RESPECTS THIS 4 RULES:

SAFETY:

CS1. ONLY ONE PROPOSED VALUE CAN BE CHOSEN.

CS2. ONLY ONE VALUE CAN BE CHOSEN.

CS3. A GOOD LEARNER CAN ONLY LEARNS ONE VALUE.

LIVENESS:

L1. A VALUE SOONER OR LATER IS CHOSEN.

- **CS1 (Validity / Proposed Value):**
  - *Meaning:* You cannot choose a value out of thin air. The chosen value must be one that was actually proposed by a participant in the system.
- **CS2 (Agreement / Consistency):**
  - *Meaning:* The system must reach a consensus on **only a single value**. Different nodes cannot decide on conflicting values (e.g., Node A cannot choose `0` while Node B chooses `1`).
- **CS3 (Integrity / Learner Accuracy):**
  - *Meaning:* A correct (non-faulty) learner can only learn a value that has officially been chosen. It protects learners from accepting unverified, fake, or unchosen data.

Summary Analogy

Think of a group of friends trying to pick a restaurant for dinner:

- **Safety (CS1–CS3):** They cannot pick a restaurant nobody suggested (CS1), they must all agree on the *same* restaurant instead of going to two different places (CS2), and no one can claim a restaurant was chosen if a consensus was never actually reached (CS3).
- **Liveness:** They cannot argue forever. Sooner or later, they *must* actually pick a restaurant and go eat (progress).

- THE SITES IN THIS PROTOCOL USE FAIL-STOP STANDARD.

Carlo Re Rome NOV. 11. 2026

- CHOSE BY MAGGIORITY.

IN PAXOS A QUORUM IS A SET OF $m - f$ ACCEPTORS WHERE

$$f = \left\lfloor \frac{m-1}{2} \right\rfloor$$

IN A GROUP OF 6 SITES $m = 6$, $f = \left\lfloor \frac{6-1}{2} \right\rfloor = 2$

THIS IS A RULE TO GUARANTEES MAGGIORITY AND STABILITY BECAUSE THE SYSTEM CAN WORK WITH f SITES DOWN.

- PAXOS STEP BY STEP

EACH PROCESS HAVE THIS STRUCTURE

- PROPOSER
  - CRND : CURRENT ROUND
  - CVAL : CURRENT VALUE
- ACCEPTOR
  - RND : ROUND
  - VRND : VOTED ROUND
  - VVAL : VOTED VALUE
- LEARNER
  - VAL : VALUE

IN THE BEGINNIG EACH OF THIS VALUES IS SET TO -1.

STEP BY STEP COMPUTATION ...

1. A PROPOSER SETS
   CRND = i
   AND SEND BRODCASTLY <PREPARE, i>
2. ACCETORS RECEIVES <PREPARE, i>
   AND IF AN ACCEPTOR HAS VRND < i IT
   SETS VRND = i
   SENDS <PROMISE, i, VRND, VVAL>
3. IF THE PROPOSER RECEIVES $M - f$ <PROMISE...>
   IT SENDS <ACCEPT, i, CVAL> TO THE ACCEPTORS
   WHERE CVAL IS :
   - A NEW CHOOSEN VALUE IF EACH PROMISSES ARRIVED HAVE VAL = -1
   - THE VAL WITH THE BIGGEST VRND

4. THE ACCEPTORS RECEIVE THE MESSAGE AND SEND ⟨LEARN, i, V⟩, WHERE CVAL = V, BROADCASTLY

5. A LEARNER THAT RECEIVES (m-f) ⟨LEARN⟩ MESSAGES SET ITS VAL = V.

- REPETITION MANAGEMENT

A ROUND IN PAXOS CAN FAILS FOR DIFFERENT CAUSES...
FOR EXAMPLE A PROPOSER CRASH DURING THE ROUND, OR AN ACCEPTOR OR COULD BE RETARDS ON THE LINKS...

- A VALUE IS CHOSEN WHEN THE MAGGIORITY OF THE ACCEPTORS VOTE IT IN THE SAME ROUND (3.)
- A VALUE IS KNOWN WHEN THE SITES RECEIVE THE MESSAGE THAT THE QUORUM IS REACHED (5.)

IF A PROPOSER DON'T KNOW IF THE VALUE IS KNOWN AND ITS TIMEOUT EXPIRES, IT OR OTHER SITES COULD SEND OTHER REQUEST TO FIX BUT WITH A CERTAIN PROTOCOL TO AVOID TO OVERWRITE AN OLD DECISION.

- FIRST RULE (ROUND DISTRIBUTION)
  THE ROUND NUMBERS IS DISTRIBUITED TO THE PROPOSA EQUALLY:
  p0 HAS 1, 4, 6...
  p1 HAS 2, 5, 7...
  p2 HAS 3, 6, 9...

TO ALSO AVOID THAT A PROPOSER IS MORE UNLUCKY THAN OTHERS...

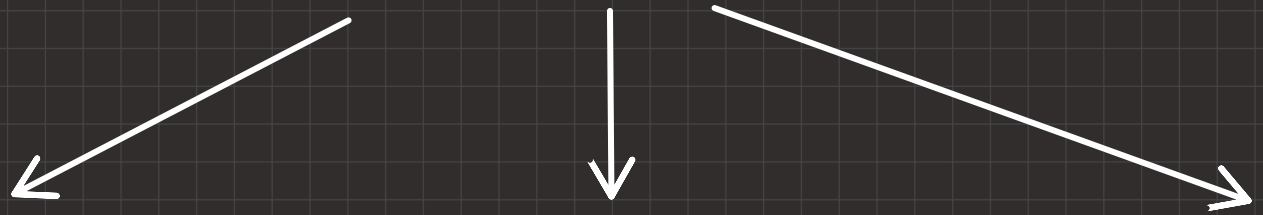
TO AVOID TIME DEPENDENCY, BECAUSE THE ROUND NUMBER IS INDIPENDENT TO TIME AND THE ROUNDS COULD BE INFINITES.

- SECOND RULE (INVENTORY)

TO AVOID TO CHOOSE A NEW VALUE AFTER CHOOSING A PREVIOUS VALUE (VIOLING CS2)
SITES HAVE INVENTORY...
WHEN A PROPOSER WANT TO PROPOSE A VALUE IT ASKS ABOUT THE LAST CHOICE AND WHEN A SITE ACCEPTOR PROMISE TO A PROPOSER TO ACCEPT ONLY MESSAGES WITH ROUND $\geq i$ IT IS BLOCKING ALL THE MESSAGES IN RETARD ...

- THIRD RULE (RECOVERY)

IF A SITE IS IN A COMPONENT DUE A PARTITION IT CAN RETRIVES INFORMATIONS IN THREE WAYS.



ATTENDING <LEARN> MESSAGES FROM THE NETWORK

ASKING TO THE OTHER SITES ABOUT THE DECISION

BECOMING A PROPOSER AN PROPOSING A VALUE FOLLOWING THE PROTOCOL.

- WHY PAXOS IS SAFE?

CS1 AND CS3 ARE SATISFIED BECAUSE:

CS 1) BECAUSE A PROPOSER CAN PROPOSE ONLY VALUES PROPOSED BY OTHER PROPOSER OR BY A CLIENT...

SAFETY:
CS1. ONLY ONE PROPOSED VALUE CAN BE CHOSEN.
CS2. ONLY ONE VALUE CAN BE CHOSEN.
CS3. A GOOD LEARNER CAN ONLY LEARNS ONE VALUE.
LIVENESS:
L1. A VALUE SOONER OR LATER IS CHOSEN.

CS3) BECAUSE A NOT CRASHED LEARNER WILL LEARN ONLY ONE VALUE IN ITS LIFE (FOR OPERATION) BECAUSE A LEARNER LEARNS A VALUE ONLY IF THE MESSAGE IS SENDED BY $(m-f)$ SITES ACCEPTORS. AND THIS APPEN WHEN THE QUORUM IS REACHED.

CS2) WE HAVE TO DEMONSTRATE IT...

CS: IF AN ACCEPTOR VOTED V IN THE ROUND i, NO ONE DIFFERENT VALUE IS CHOOSEN IN A PREVIOUS ROUND

WE HAVE TO DEMONSTRATE CS TO DEMONSTRATE CS2

1. Q IS THE QUORUM OF PROMISES OF THE ROUND i AND J $(J<i)$ IS THE BIGGEST ROUND ASSOCIETED TO A VOTE IN Q

EXAMPLE

(<PROMISE, 1, V> ; <PROMISE, 3, V> ; <PROMISE, 5, V>
<PROMISE, 2, V> ; <PROMISE, 4, V> ;

THIS IS J

2. IN THE ROUNDS $(J+1)$ AND $(i-1)$ NO ONE VALUES IS CHOSEN BECAUSE ACCEPTORS IN Q DON'T VOTED (BECAUSE THEY PROMISED THIS) AND THE ACCEPTORS NOT IN Q ARE NOT A MAGGIORITY...

3. IN THE ROUND J THE ONLY POSSIBLE VALUE WAS V.
4. BY REVERSE INDUCTION WE CAN ARRIVE FROM J TO ROUND 0.
5. CS → CS2

- IF MORE THAN $\frac{f}{2}$ ACCEPTORS FAILS, PAXOS DON'T CHOSES INCONSISTENT OR WRONG VALUES, IT DON'T CHOSES VALUES.

- LIVENESS

LET'S SUPPOSE THAT WE HAVE 3 PROPOSERS...

p_1 SENDS <PREPARE, 1>
p_2 SENDS <PREPARE, 2>
p_3 SENDS <PREPARE, 3>
p_1 SENDS <PREPARE, 4>
[...]

WHEN p_1 SENDS <PREPARE, 1> AND THE ACCEPTORS ACCEPT IT AND SEND <PROMISE, 1, ..., ...> WHEN ARRIVES <PREPARE, 2> THEY SEND <PROMISE, 2, ..., ...> BECAUSE $2 \geq 1$

- FLP THEOREM

* IN A DISTRIBUTED SYSTEM IF FAILES ARE POSSIBLE, NO ONE DETERMINISTIC ALGORITHMS CAN ENSURE SAFETY AND LIVENESS.
* THIS IS TRUE BECAUSE IN A DISTRIBUTED SYSTEM IS IMPOSSIBLE TO KNOW IF A PROCESS IS SLOW, WITH NETWORK ISSUES OR BROKEN
* FLP THEOREM IS INEVITABLE, PAXOS PREFER SAFETY TO LIVENESS.

- A SIMPLE SOLUTION
  - LEADER

EACH NODE HAS THE MODULE SL (FAILURE DETECTOR) BASED ON TIMEOUTS AND IT CONTAINS WHICH ONE IS THE PROPOSER LEADER, A PROPOSER THAT HAS PRIORITY ON THE OTHERS BUT DUE NETWORK PROBLEMS WE COULD HAVE PROBLEMS, 2 PROPOSER LEADER FOR EXAMPLE... BUT IF NETWORK IS STABLE THE LIVENESS IS RESPECTED.

CH. 8 MULTIPAXOS

Carlo Re Rome NOV. 12. 2026

- IF WE WANT TO MODIFY A DATA ON A DISTRIBUTED SYSTEM WHERE ITS SITES HAVE SAME DATA INSIDE THE EASLY SOLOUTION IS TO SEE IT AS A REPLICATED STATE MACHINE BECAUSE:

- TO DECIDE THE SEQUENCE OF OPERATIONS THIS ALGORITHM USES A PAXOS EXECUTION FOR EACH CELL...
- IF CELLS 7 AND 9 ARE DECIDED, THE SERVERS DON'T EXECUTE THE OPERATION 9 IF THEY DON'T HAVE CELL 8.
- THE LEADER IS STABLE BECAUSE:

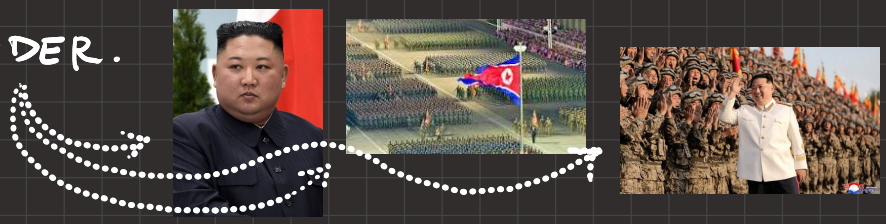
IF ARRIVE 3 REQUESTS (a, b, c) FROM 3 DIFFERENT SERVERS

- WITHOUT COORDINATOR

WE COULD HAVE THE SAME PROBLEM OF PAXOS, PROCESSES THAT BLOCK EACH OTHER...

- WITH A STABLE COORDINATOR

THE COORDINATOR ASSIGNES EACH CELL TO A REQUEST. ACCEPTORS VOTE AND CONFIRM THE ORDER DECIDED BY THE LEADER.



- IMAGINE USING ALL THE MESSAGES USED IN SIMPLE PAXOS IN THIS MULTIPAXOS, TO DECIDE EACH CELL THE ALGORITHM HAS TO USES 4 MESSAGES

<PREPARE> → <PROMISE> → <ACCEPT> → <LEARN>

BUT THIS TOO LONG AND SOO SLOW...

- THE SOLUTION IS TO SEND 1 <PREPARE> MESSAGE PER SEQUENCE BY THE LEADER AND OBTAIN <PROMISE> MESSAGES IN THE FIRST ROUND ONLY, AFTER THIS PRELIMINAR OPERATIONS THE SYSTEM EXCHANGES ONLY <ACCEPT> AND <LEARN> AND <PROPOSE> MESSAGES.

BECAUSE OTHERS ARE USELESS...

When the leader has to manage a sequence of future operations (slots/instances 8, 9, 10, 11, and so on), repeating `PREPARE` and `PROMISE` for every single slot is wasteful for three reasons:

- **No one has voted in future slots yet (Empty inventory):** Slots 10, 11, or 100 are in the future, so nobody has worked on them yet. If the leader sent a `PREPARE` for slot 10, one for 11, and one for 12 asking *"what have you voted here?"*, the acceptors would always reply *"nothing"* (`-1`) for every single one. Asking a thousand identical questions only to hear *"no vote"* a thousand times is pointless: asking once for all free slots is enough.
- **The promise does not depend on the slot (Universal promise):** When an acceptor replies `PROMISE` to round i, it makes this pledge: *"I will no longer vote in rounds lower than i"*. This rule checks **only the round number** i, not the instance/slot number. Therefore, a single promise is enough to block old rounds simultaneously for slot 8, slot 9, slot 10, and all future slots, as long as the leader stays the same.
- **The user's command is not needed yet (Value independence):** The $\langle \text{PREPARE}, i \rangle$ message **does not contain the operation to execute** (such as *"rename the file to Tesi.pdf"*) . The actual operation is sent only later, inside the $\langle \text{ACCEPT}, \text{instance}, i, x \rangle$ message. As a result, the leader can execute `PREPARE` and `PROMISE` **in advance**, even before users send any requests.

It is just like **taking attendance once at the opening of a meeting**: that single roll call applies to every item on the agenda (all future slots), and then for each individual item you proceed directly to the vote (`ACCEPT` and `LEARN`).

Would you like to go through how Table 9.1 (an acceptor's internal state) gets filled out to see this mechanism visually?

- EXAMPLE OF MULTIPAXOS EXECUTION...

1. THE LEADER SENDS <PREPARE, i> MESSAGES AND WAITS FOR <PROMISE, i> MESSAGES ... WHEN IT HAS A QUORUM IT CAN START TO RECEIVES PROPOSALS
2. IF ANNA FOR EXAMPLE WANT TO PROPOSE A CHANGE FOR A FILE (x), ANNA SENDS THIS PROPOSAL TO HER PROXY SERVER (S_3 FOR EXAMPLE) AND THIS SERVER WILL SENDS THIS PROPOSAL TO THE LEADER USING <PROPOSE, x>
3. THE LEADER RECEIVE <PROPOSE, x>, TAKE THE FIRST FREE POSITION IN THE SEQUENCE AND SENDS TO THE ACCEPTORS <ACCEPT, 8, i, x>
   ↑ for example
4. THE ACCEPTORS RECEIVE THE MESSAGE AND SEND <LEARN, 8, i, x> TO THE LEARNERS.
5. WHEN A LEARNER RECEIVE A (m-f) REQUEST <LEARN...> ACCEPT THE VALUE.

- IF THE LEADER RECEIVE A SECOND REQUEST IT DON'T WAIT FOR THE TERMINATION OF THE LAST ONE BUT IT JUST SEND THE OTHER REQUEST
- IF IN A CERTAIN POINT OF THE COMPUTATION SOME CRASHES, THE LEADER CAN RE-SEND THE SAME <ACCEPT...> MESSAGE AND COMPLETE THE OPERATION.
- A ROUND CONTAINS MORE ISTANCES, AN ISTANCE IS 1 CICLE TO FILL A CELL OF THE SEQUENCE.

THE ISTANCES HAS ONLY THE ROUND NUMBER `rnd` AND `vrnd` IN COMMON BUT THE OTHERS VALUE ARE DIFFERENT.

EXAMPLE...

istance.	rnd	vrnd	vval
8	i	i	x
9	i	i	y
10, 11, ...	i	-1	-1

THIS PROTOCOL WORKS WELL UNTIL THE LEADER CRASHES

- ELECT A NEW LEADER.

- THE SIMPLE WAY TO KNOW IF A LEADER IS DEATH AND REPLACE IT IS USING HEARTBEAT MESSAGES... ALL THE PROCESSE SENDS THIS MESSAGES EACH OTHER COSTANTLY, IF LEADER DOESN'T SEND HEARTBEAT , THE LEADER IS CONSIDERED DEAD AND IT WILL BE REPLACED WITH THE PROCESS WITH SMALLEST ID
- INSTABILITY ON THE NETWORK CAN CAUSE FAKE DEAD OR MULTILEADER SITUATION BUT
  1. COERENCE IS PRESERVED BECAUSE THE NEW LEADER HAS A NEW vrnd
  2. WHEN THE NEW LEADER RECIEVE THE HEARTBEAT MESSAGES FROM THE OLD LEADER IT COMPREENDS THE SITUATION AND ADJUST THE HEARTBEAT TIMEOUT TO AVOID MISUNDERSTANDING.

- WHEN A NEW LEADER SETTLES IN...
  1. SENDS <PREPARE, i'> WHERE $i' > i$ FOR EACH ALREDY OPENED ISTANCE
  2. WHEN IT RECEIVE <PROMISE ...> FROM A MAGGIORITY FOR EACH ALREDY OPENED INSTANCE RE-SENDS THE SAME CONTENT OF THE ISTANCE BUT WITH rmd i'.
     IF AN ISTANCE DOESN'T HAVE VOTES, THE LEADER CLOSE IT SET IT AS NO-OP.
     THIS REQUEST WILL BE RESENDED AFTER THE EXPIRATION OF A TIMEOUT.